PROJECT PROPOSAL SYNOPSIS

Formwell

Personal Fitness Tracking and Activity Monitoring System

A synopsis submitted in partial fulfilment of the requirements for

[Program / Degree Name]

Submitted by:	[Student Name]
Register / Roll No.:	[Registration Number]
Project guide:	[Guide Name and Designation]
Department:	[Department Name]
Institution:	[Institution Name]
Academic year:	[Academic Year]

October 2026

Abstract

Formwell is a browser-based fitness tracking application supported by a Go HTTP API and a PostgreSQL database. It provides account creation and sign-in, profile-baseline display, predefined cardio, strength and meal-plan views, daily water and step entry, exercise-progress logging, and date-selected progress review. The application uses a standalone HTML, CSS and JavaScript frontend; the backend uses Go, GORM and PostgreSQL. A JWT bearer token is issued at sign-in and is used by the frontend for protected requests.

This synopsis documents the code-backed project as it exists in the accompanying repositories and separates current capabilities from the recommended work needed for a production-ready proposal. The data model, request flow, module boundaries, deployment topology, functional requirements, project schedule, and security considerations are described with diagrams and tables. The project is categorized primarily as an RDBMS-backed information system, with object-oriented/data-modeling and web-networking elements. It is not an AI or machine-learning system: the visible chat screen is a disabled user-interface preview and has no connected conversational service.

The implementation is suitable as a demonstrator of full-stack web development and relational persistence. It is not a clinical or medical decision-support tool. The code review also identifies important limitations that should be addressed before real personal data is used, particularly plaintext password persistence, exposure of the password field in profile responses, inconsistent ownership enforcement, localStorage token storage, and HTTP-only local deployment assumptions. Student, guide, institutional, and academic-year information remains a placeholder pending confirmation.

Document Status

This document is based on the sibling repositories `frontend-fitness` and `backend-fitness` as inspected for this draft [[6, 7]]. Statements marked **Implemented** describe behavior present in source. Statements marked **Recommended** or **Proposed** describe work not claimed to be complete. Schema details describe the expected schema implied by GORM models, migration calls, tags, and explicit SQL; the live database was not independently introspected. Hardware sizing and the eight-week schedule are planning assumptions, not measurements or a committed delivery plan.

Contents

1 Introduction and Objectives **1**
- 1.1 Introduction . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Objectives . . . 1
- 1.4 Success Criteria . . . 1

2 Project Category, Scope, and Background **1**
- 2.1 Project Category . . . 1
- 2.2 Scope of the Solution . . . 2
  - 2.2.1 Included in the current implementation . . . 2
  - 2.2.2 Excluded or not yet implemented . . . 2
- 2.3 Assumptions and Constraints . . . 2

3 Literature Review and Requirements **2**
- 3.1 Literature Review . . . 2
- 3.2 Problem Definition . . . 2
- 3.3 Functional Requirements . . . 3
- 3.4 Non-Functional Requirements . . . 3

4 Technical Specifications and Architecture **4**
- 4.1 Tools and Platforms . . . 4
- 4.2 Hardware and Software Requirements . . . 4
- 4.3 System Context and Overall Architecture . . . 4
- 4.4 Level-2 Data Flow: Daily Activity . . . 5

4.5 API Surface . . . 5

5 Analysis and Data Models **6**
5.1 Entity-Relationship Model . . . 6
5.2 Activity Diagram . . . 7
5.3 Class and Component Diagram . . . 7
5.4 State Diagram . . . 8

6 Database Design **8**
6.1 Design Overview . . . 8
6.2 User and Daily Activity Tables . . . 8
6.3 Lookup and Mapping Tables . . . 9
6.4 Schedule and Diet Tables . . . 9
6.5 Relationship Cardinality, Integrity, and Normalization . . . 9

7 Module Design and Implementation Methodology **10**
7.1 Module Inventory and Effort Estimate . . . 10
7.2 Process Logic by Module . . . 10
7.2.1 M1: Identity and session . . . 10
7.2.2 M2 and M3: Profile, dashboard, and plan catalogue . . . 11
7.2.3 M4 and M5: Activity logging and history . . . 11
7.2.4 M6–M9: API, persistence, verification, and delivery . . . 11
7.3 Data Structures . . . 11
7.4 Reports Likely to Be Generated . . . 11

8 Project Planning and Scheduling **12**
8.1 Planning Assumptions . . . 12
8.2 Gantt Schedule . . . 12
8.3 PERT Network . . . 12
8.4 Risks and Mitigations . . . 14

9 Network Architecture and Security **14**
9.1 Overall Network Architecture . . . 14
9.2 Security Mechanisms: Current State and Required Work . . . 14
9.3 Security Requirements by Level . . . 15

10 Future Scope, Conclusion, and Bibliography **15**
10.1 Future Scope and Enhancements . . . 15
10.2 Conclusion . . . 15

List of Figures

1 Level-0 data-flow diagram (system context). . . . 5
2 Level-1 data-flow diagram showing the main processes and data stores. . . . 5
3 Level-2 data-flow diagram for writing daily activity. . . . 5
4 ER model of declared relationships and code-driven plan selection. . . . 6
5 Activity diagram for the signed-in tracking workflow. . . . 7
6 High-level class and component relationships. . . . 7
7 State diagram for a user-day activity record. . . . 8
8 Proposed eight-week Gantt schedule. . . . 12
9 PERT precedence network for the proposed baseline schedule. . . . 12
10 Current local network architecture and trust boundaries. . . . 13

List of Tables

1 Related guidance and its relevance to the project . . . 3
2 Functional requirements and status . . . 3
3 Current tools and deployment assumptions . . . 4
4 Principal API routes visible in source . . . 6
5 Core user and activity schema . . . 8
6 Lookup and category-mapping tables . . . 9
7 Module effort estimate and implementation status . . . 10
8 Principal data structures by layer . . . 11
9 Project risks and practical responses . . . 13
10 Security controls, gaps, and recommendations . . . 14

1 Introduction and Objectives

1.1 Introduction

People who train independently often need a simple way to keep baseline profile details, daily activity, and suggested routines together. Formwell addresses this information-management problem through a signed-in browser interface that communicates with a small API and a relational database. It centralizes profile data, preset exercise and meal plans, daily hydration and step measurements, and exercise completion values in a single application.

The product name shown in the frontend is Formwell. The application is a learning and project-proposal system, not a verified fitness prescription. Its plan selection is deterministic and based on profile values and seeded lookup content. No evidence in the repositories demonstrates clinical validation, synchronization with wearable devices, multi-tenant deployment, or an AI service. Any public or academic presentation should preserve those boundaries.

1.2 Problem Statement

A user tracking exercise in separate notes or spreadsheets can find it difficult to relate daily measurements to a suggested routine and later review. The project provides one account-based interface for recording and viewing a narrow set of fitness-related values. The current implementation has a working vertical slice across browser forms, HTTP handlers, database persistence, and selected-day summaries; it does not yet provide a normalized, strongly validated, production-secure health-data platform.

1.3 Objectives

The project objectives are:

1. Provide a browser workflow for account creation and sign-in, with a profile baseline consisting of age, height, weight, gender, and BMI.
2. Retrieve predefined cardio, strength, and meal-plan content selected by the backend from age-group and BMI-derived categories.
3. Allow a signed-in user to record daily water intake, step count, and completion counts for supported exercises.
4. Persist daily activity by user and date, and retrieve it for a selected date.
5. Present profile, current-day metrics, training targets, and history in distinct frontend screens.
6. Demonstrate a modular client/API/database architecture using browser technologies, Go, JWT bearer authentication, GORM, and PostgreSQL.
7. Identify and prioritize security, data-integrity, usability, and maintainability improvements before any production deployment.

1.4 Success Criteria

The proposed acceptance checks are functional rather than performance claims: a new account can be created; duplicate usernames are rejected; valid credentials return a token; an invalid or expired token is rejected by protected routes; profile and plan views render for a valid account; daily metrics and exercise values can be written and read by date; a user's records cannot be read or changed by another account; invalid values are rejected; and sensitive fields are absent from public responses. The first six describe visible workflows or target behavior; the final four require security and validation work beyond the present code. There are no repository test files or measured performance baselines at the time of this synopsis.

2 Project Category, Scope, and Background

2.1 Project Category

The primary category is **RDBMS-based application development**: PostgreSQL stores user profiles, daily activity, lookup values, and predefined plans. The project also falls under **object-oriented and modular application development** because the Go implementation models users, workout plans, handlers, and a database interface, and under **web networking** because a browser exchanges JSON over HTTP with an API. It is not a mobile application: the frontend is a standalone responsive website. It is not artificial intelligence, machine learning, expert systems, or data science; the seeded plan selection is rule-based and the chat is explicitly preview-only.

2.2 Scope of the Solution

2.2.1 Included in the current implementation

- Sign-up form with personal baseline values and client-side BMI calculation, plus login and logout screens.
- JWT-based sign-in and middleware-protected API routes.
- Profile, dashboard, workout-plan, daily-statistics, and selected-date report screens.
- Preseeded workout schedule, exercise target, and diet-plan data selected through code-defined categories.
- Daily persistence of water, steps, and exercise counts under a username/date combination.
- Local development deployment with a static file server, Go API, and PostgreSQL container.

2.2.2 Excluded or not yet implemented

There is no profile-edit workflow, role or administrator console, custom routine authoring, downloadable or scheduled report, wearable/device integration, payment, mobile-native client, notification service, clinical review, AI chat, analytics pipeline, or verified production deployment. Chat is a disabled visual preview. Data is limited to the fields modeled in the application. The application does not establish that its targets are appropriate for any individual's health condition; users should not interpret them as medical advice.

2.3 Assumptions and Constraints

The report assumes a single application instance and a PostgreSQL database reachable on a trusted development network. Local README instructions use API port 8080, frontend port 8081, and a Docker-published database port 5433 mapped to container port 5432. The API defaults to a development database password and disables PostgreSQL TLS in its DSN; these are not acceptable production defaults. The current database field constraints are uneven, plan data is seeded into category-specific tables, and several categories are represented by integer selectors. The browser and server currently use local HTTP URLs and a localStorage-held token. These constraints must be addressed before broader deployment.

3 Literature Review and Requirements

3.1 Literature Review

The World Health Organization's 2020 guidance presents physical activity and sedentary behavior as public-health concerns and gives evidence-based recommendations for population groups [[1]]. For this project, its relevance is contextual: recording activity can support personal awareness, but this application has not implemented or clinically validated the guideline recommendations. The system therefore avoids translating its seeded targets into medical claims.

The JWT specification defines a compact claims representation and registered claims including expiration [[2]]. Formwell uses an HMAC-SHA-256-signed token carrying a username and expiry, which gives the API a stateless bearer credential for the current prototype. The specification itself does not provide authorization policy, secure storage, transport encryption, or safe user-data scoping; those are application responsibilities. The implementation currently demonstrates token validation but does not consistently bind a path username to the authenticated subject.

GORM's migration documentation describes schema creation and evolution from Go models [[3]]. The project uses AutoMigrate and model tags, which reduces manual schema setup for a demonstration, but model-derived migration is not a substitute for reviewed, versioned migrations or direct inspection of deployed schema. PostgreSQL constraints provide mechanisms for primary keys, foreign keys, uniqueness, and nullability [[4]]. The current schema uses several of these mechanisms, while leaving validation rules such as positive metric values and date requirements to client-side input controls or application code. Those rules should be enforced on the server and, where appropriate, in the database.

For credential storage, OWASP recommends adaptive password hashing rather than plaintext persistence and identifies modern choices such as Argon2id, scrypt, bcrypt, or PBKDF2 according to deployment requirements [[5]]. The present backend stores and compares the submitted password directly, so it does not meet that baseline. This observation is an implementation finding, not an assertion that hashing alone resolves authentication risks.

3.2 Problem Definition

The technical problem is to maintain a coherent user-specific history while providing a low-friction browser workflow for logging. The application must identify the account, select targets, accept daily values, persist an idempotent user/date record, and retrieve the record for presentation. Correctness depends on the identity used

Table 1: Related guidance and its relevance to the project

Source	Principle used in this synopsis	Implication for Formwell
WHO physical-activity guideline	Activity tracking belongs in a health context and should not overstate evidence or personalization.	Describe the system as a logging and awareness tool, not clinical advice.
JWT standard	Signed token claims require validation and an explicit application trust policy.	Keep the expiry check; strengthen subject scoping, key management, and transport security.
GORM migration documentation	Models and tags can drive migration, but schema change is an operational responsibility.	Document generated constraints accurately and move toward reviewed migration history.
PostgreSQL constraint documentation	Database constraints enforce referential integrity and data validity.	Add non-null, range, check, and foreign-key rules where requirements demand them.
OWASP password-storage guidance	Passwords should be stored as adaptive hashes, not recoverable plaintext.	Replace plaintext storage and comparison with a vetted password-hashing implementation.

for authorization, consistent date handling, matching UI fields to API model fields, and database constraints. Security depends on server-side authentication and authorization, safe credential handling, minimizing sensitive output, and secure deployment configuration.

3.3 Functional Requirements

Table 2: Functional requirements and status

ID	Requirement and status
FR-01	The system shall accept a profile and unique username at sign-up. Current: implemented; server-side field validation and robust duplicate/error handling need improvement.
FR-02	The system shall authenticate a username/password and issue a time-limited bearer token. Current: implemented; plaintext credentials are a critical gap.
FR-03	The system shall display the authenticated user's profile baseline. Current: implemented; response minimization is required because the current user model includes a password field.
FR-04	The system shall select and display cardio, strength, and diet information for the profile category. Current: implemented using seeded tables and code-defined selection.
FR-05	The system shall display daily hydration, steps, and exercise completion against targets. Current: implemented for current-day data.
FR-06	The user shall be able to submit water intake and step count. Current: implemented through the daily-input form and write endpoint.
FR-07	The user shall be able to submit supported exercise completion counts. Current: implemented through cardio and strength forms.
FR-08	The system shall retrieve a saved activity record for an explicitly selected date. Current: implemented by date-based endpoints and the reports view.
FR-09	The user shall be able to sign out and clear browser-held session values. Current: implemented by removing localStorage entries.
FR-10	The system shall prevent access to another user's profile and activity. Current: incomplete; route identifiers are not consistently checked against token claims.
FR-11	The system shall offer AI-generated coaching or an interactive chat. Current: not implemented; the chat is a static UI preview.
FR-12	The system shall export reports as PDF or CSV. Current: not implemented; selected-day information is rendered in-app only.

3.4 Non-Functional Requirements

- **Security:** use adaptive password hashes, server-side validation, subject-based authorization, TLS in deployed environments, least-privilege database credentials, and secret management. These are target requirements, not

all current controls.

- **Integrity:** preserve one daily record per user/date; enforce valid ownership, date format, and non-negative numeric values at the API and database boundary.
- **Usability:** keep authentication, dashboard, workouts, stats, reports, and profile navigation understandable on desktop and mobile screen sizes.
- **Maintainability:** separate presentation, HTTP handling, business rules, and persistence; document schemas and API contracts; add automated tests and versioned migrations.
- **Portability:** use supported Go, PostgreSQL, and browser runtimes; keep deployment settings configurable through environment variables.
- **Privacy:** return only fields required by the requested screen, avoid logging credentials or bearer tokens, and define retention and deletion behavior before real users are onboarded.

4 Technical Specifications and Architecture

4.1 Tools and Platforms

Table 3: Current tools and deployment assumptions

Layer	Current implementation	Requirement / assumption
Frontend	HTML, CSS, JavaScript; static files served locally; browser fetch and localStorage.	Current evergreen desktop/mobile browser. Static hosting must use HTTPS outside local development.
API	Go module declaring Go 1.22.1; standard HTTP server and ServeMux; CORS middleware.	Go 1.22 or later as stated in README; restrict CORS to known origins.
Authentication	github.com/dgrijalva/jwt-go v3.2.0; HS256 bearer token.	Secret supplied via JWT_SECRET; replace deprecated dependency with a maintained JWT library as a proposed maintenance task.
Persistence	PostgreSQL driver and GORM v1.25.12; AutoMigrate and seed data.	PostgreSQL 16 container for local development; use reviewed migrations for production.
Container	Docker multi-stage API image and PostgreSQL Compose service.	Docker Engine and Compose for the documented local database setup.
Development client	Python 3 static file server is documented for local frontend hosting.	Development convenience only; production static hosting should be a hardened web server/CDN.

4.2 Hardware and Software Requirements

The repositories do not define measured minimum hardware. The following is a proposed small-development baseline, not a performance guarantee: a 64-bit computer with a modern dual-core processor, 8 GB memory, several gigabytes of free disk for source, container images, and database volume, and a network interface for localhost services. A developer environment requires macOS, Windows, or Linux with Go 1.22+, Docker with Compose, PostgreSQL-compatible client tools if database inspection is needed, and a current browser. A deployment environment additionally requires a supported Linux host or managed container service, persistent database storage, TLS termination, backups, monitoring, and secret storage. Capacity must be measured against a defined workload before production sizing.

4.3 System Context and Overall Architecture

The system uses a three-part logical architecture: browser client, Go API, and PostgreSQL persistence. The frontend is served independently from the API. The API opens the database connection and performs migrations and seed insertion at startup. The browser sends JSON to API routes; protected requests include a bearer token. Local configuration uses HTTP and localhost endpoints.

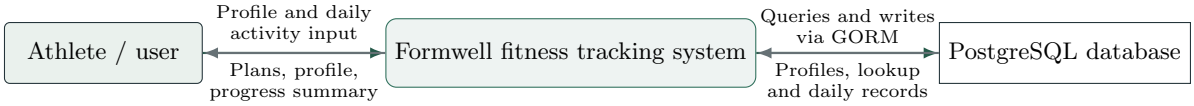


Figure 1: Level-0 data-flow diagram (system context).

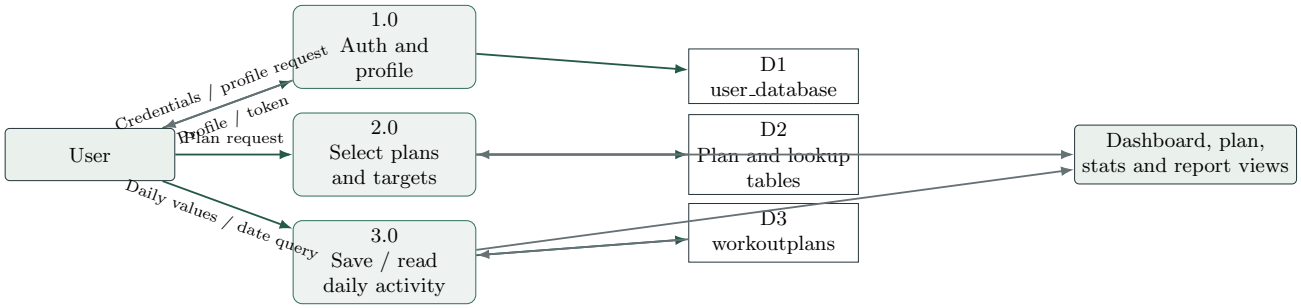


Figure 2: Level-1 data-flow diagram showing the main processes and data stores.

4.4 Level-2 Data Flow: Daily Activity

The daily-entry workflow is decomposed below. The current middleware validates the bearer token, but the service should additionally derive the owner from the token rather than trusting the username in the path. Server validation and error handling are required before persistence. The diagram distinguishes the current request flow from the authorization rule that is a required improvement.

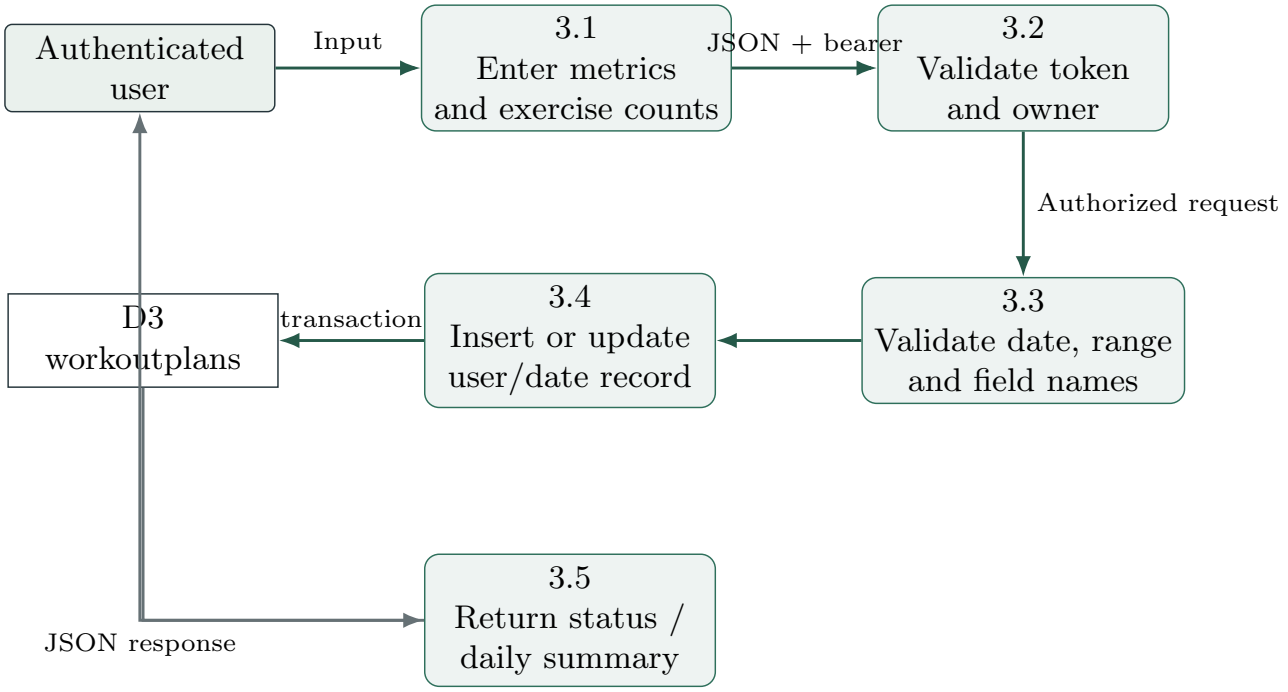


Figure 3: Level-2 data-flow diagram for writing daily activity.

4.5 API Surface

The frontend uses POST requests for its JSON request helper, including reads; the Go handlers are registered with generic ServeMux handlers rather than method-specific route declarations. The table summarizes routes from the current server wiring and frontend call sites. Route authorization annotations refer to middleware wrapping in `main.go`; they are not a substitute for per-record ownership checks.

Table 4: Principal API routes visible in source

Route	Method used by UI	Middleware	Purpose
`/signup`	POST	None	Create a user profile.
`/login`	POST	None	Verify credentials and issue a JWT; also initializes today's targets.
`/check-auth`	GET	Auth middleware	Frontend session check.
`/getuser/{id}`	POST	Auth middleware	Load a profile. Current response is the full user model.
`/getuser/{id}/workoutplan`	POST	Auth middleware	Load strength targets.
`/getuser/{id}/cardioplan`	POST	Auth middleware	Load cardio targets.
`/getuser/{id}/dietplan`	POST	Auth middleware	Load a diet plan.
`/userinput/{id}`	POST	Auth middleware	Insert or update today's submitted values.
`/userdailydata/{id}`	POST	Auth middleware	Read today's daily record.
`/userdailydata/{id}/{date}`	POST	Auth middleware	Read daily measurements for a date.
`/userdailydatatarget/{id}`	POST	Auth middleware	Read today's exercise targets and completion values.
`/userdailydatatarget/{id}/{date}`	POST	Auth middleware	Read targets and completion values for a date.
`/refresh` and `/home`	Not used by UI	Handler-level / middleware differs	Token refresh and protected demonstration endpoint; refresh is not wired through the common middleware.

5 Analysis and Data Models

5.1 Entity-Relationship Model

Figure 4 shows declared database relationships. Solid relationships correspond to explicit primary/foreign-key declarations in source, including the manually ensured user-to-workout foreign key. Dashed relationships indicate selector logic in application code and are not foreign keys to schedule or diet tables. The ER figure omits repeated weekday columns for readability; they are listed in the schema inventory in Chapter 6.

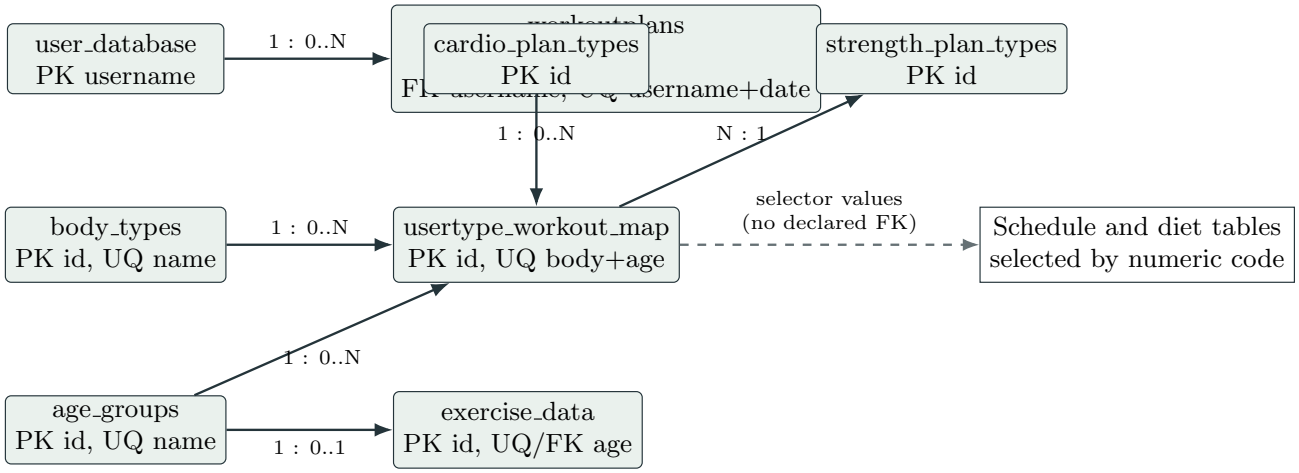


Figure 4: ER model of declared relationships and code-driven plan selection.

5.2 Activity Diagram

The activity model describes the primary user journey. Error branches return the user to a form with a message; the report is a selected-day screen, not a downloadable artifact.

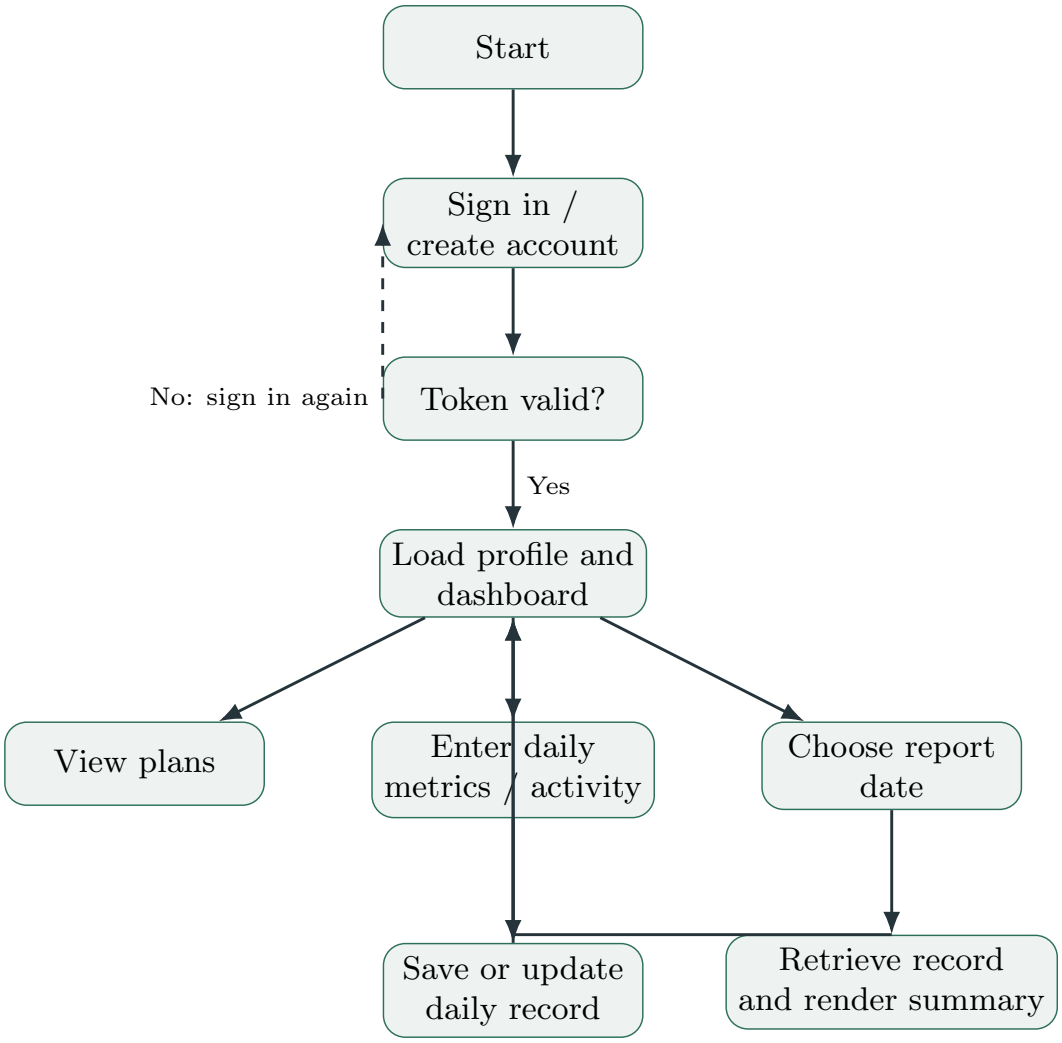


Figure 5: Activity diagram for the signed-in tracking workflow.

5.3 Class and Component Diagram

The diagram is a conceptual view of the implemented package boundaries, not a UML claim that every Go type has a corresponding frontend class. The frontend is procedural JavaScript organized around rendering and request functions. The backend exposes handlers that depend on a database interface, with the concrete implementation using GORM.

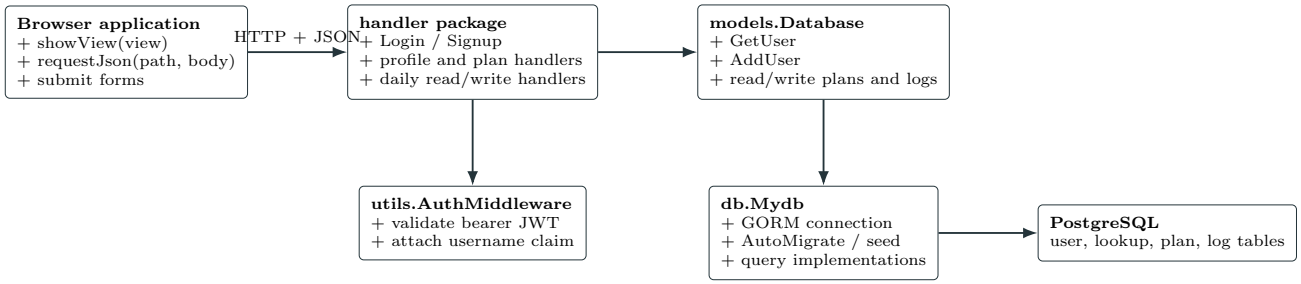


Figure 6: High-level class and component relationships.

5.4 State Diagram

The current daily record follows an insert-or-update pattern when the user submits values. A record is identified by username and the server's current date for writes. The desired authorization state is shown as an explicit guard; the current application must be amended to make the authenticated subject authoritative.

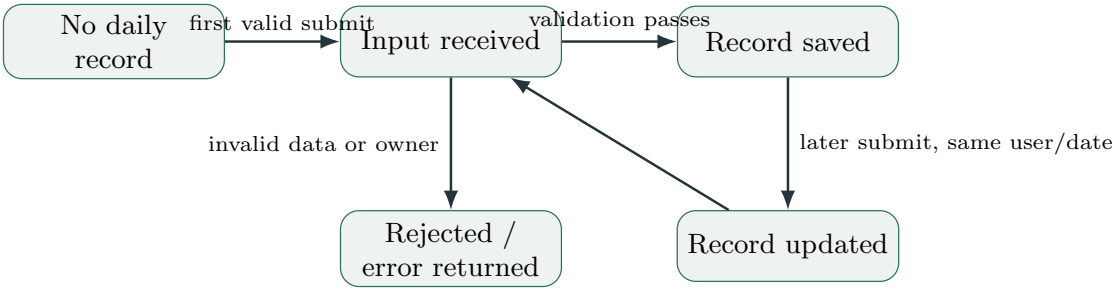


Figure 7: State diagram for a user-day activity record.

6 Database Design

6.1 Design Overview

The database is created at API startup through GORM AutoMigrate calls and seeded with category lookup values, schedules, exercise targets, and meal-plan rows. Source declares 17 application tables. There are no explicit audit timestamps in the model structs. The expected SQL types below use the Go model types and PostgreSQL/GORM mapping; inspect a running database before treating generated physical types or existing migrated state as independently verified. Primary keys, unique indexes, foreign keys, nullability, and size tags are stated where declared in source.

6.2 User and Daily Activity Tables

Table 5: Core user and activity schema

Table / key	Columns and declared constraints
`user_database`	
PK: `username`	`username`: string primary key; `firstname`, `lastname`, `password`, `gender`: strings; `age`, `height`, `weight`, `bmi`: 64-bit integers. The primary key is not-null and unique. No other explicit not-null, range, or uniqueness constraints are declared. Password is currently stored as supplied.
`workoutplans`	
PK: `id`	`id`: unsigned integer primary key; `date`: string; `username`: string, not-null; composite unique index `idx_user_date` across username and date. Named FK `fk_workoutplans_user`: username references `user_database.username`, ON UPDATE CASCADE and ON DELETE CASCADE.
`workoutplans`	
activity columns	Integer completion fields: `squatsdone`, `deadliftdone`, `pushupsdone`, `pullupsdone`, `jumpingjacksdone`, `weightliftdone`, `benchpressdone`, `lungesdone`, `legpressdone`, `runningdone`, `cyclingdone`, `swimmingdone`, `walkingdone`. Integer daily measures: `water`, `steps`. Source declares no not-null or non-negative constraints for these fields.
`workoutplans`	
target columns	String target fields: `squats`, `deadlift`, `pushups`, `pullups`, `jumpingjacks`, `weightlift`, `benchpress`, `lunges`, `legpress`, `running`, `cycling`, `swimming`, `walking`. No explicit not-null, size, or check constraints are declared.

The composite unique index expresses one non-null username/date pair per user, but the date field is not explicitly declared NOT NULL. In PostgreSQL, unique constraints permit multiple rows with a null component under the default null-distinct behavior; therefore the intended invariant should be strengthened with a NOT NULL date constraint. The model stores dates as strings rather than a date type. Source code also computes a package-level date once at process startup, which can become stale if the process crosses local midnight. Recommended design: use a database DATE or a consistently parsed ISO date, calculate the date at request time in a declared timezone, and enforce a non-null user/date unique key.

6.3 Lookup and Mapping Tables

Table 6: Lookup and category-mapping tables

Table	Columns and declared constraints
`body_types`	`id`: integer primary key; `name`: string NOT NULL and unique. Seed IDs 1–3 with names `type_1`–`type_3`.
`age_groups`	`id`: integer primary key; `name`: string NOT NULL and unique. Seed IDs 1–3 with names `group_1`–`group_3`.
`cardio_plan_types`	`id`: integer primary key. Seed IDs 1–3.
`strength_plan_types`	`id`: integer primary key. Seed IDs 1–3.
`usertype_workout_map`	`id`: unsigned integer primary key; `body_type_id`, `age_group_id`, `cardio`, and `workout`: integers, all NOT NULL. Composite unique index `idx_body_age` across body type and age group. Declared foreign keys: body type ID to `body_types.id`, age group ID to `age_groups.id`, cardio selector to `cardio_plan_types.id`, and workout selector to `strength_plan_types.id`; update cascades and delete restricts are specified in model tags.
`exercise_data`	`id`: unsigned integer primary key; `age`: integer NOT NULL and unique, declared as a foreign key to `age_groups.id` with update cascade/delete restrict. Thirteen string target fields, each NOT NULL: `running`, `cycling`, `swimming`, `walking`, `squats`, `deadlift`, `pushups`, `pullups`, `jumpingjacks`, `weightlift`, `benchpress`, `lunges`, `legpress`.

6.4 Schedule and Diet Tables

The cardio schedule tables are `workout_schedule1`, `workout_schedule2`, and `workout_schedule3`; the strength schedule tables are `strength_schedule1`, `strength_schedule2`, and `strength_schedule3`. Each has `id` (unsigned integer primary key) and seven string columns `monday`, `tuesday`, `wednesday`, `thursday`, `friday`, `saturday`, and `sunday`, each NOT NULL. Each category table is seeded with three rows. These wide weekday rows are schedule catalogues; they are not user-owned history records and no direct FK from the category map to a particular schedule row is declared.

There are three diet tables: `diet_plan1`, `diet_plan2`, and `diet_plan3`. Each contains `day` (string primary key and NOT NULL), plus `breakfast`, `lunch`, and `dinner` (strings tagged with size 255, with no explicit NOT NULL declaration). Each table is seeded with seven weekday rows. The source currently seeds the same meal text in each table; table selection is based on category code. This duplication is a maintainability concern and can be replaced by a single normalized plan table after a migration plan is agreed.

6.5 Relationship Cardinality, Integrity, and Normalization

One user can have zero or many daily records; each daily record must refer to one user through the named foreign key. One body type and one age group can each participate in zero or many category mappings. Each mapping row refers to one body type, age group, cardio selector, and strength selector. An age group can be associated with at most one exercise-data row because `exercise_data.age` is unique. Schedule and diet associations are currently application selection rules, not relational constraints. Repeated plan tables and weekday-as-column representation are not fully normalized for flexible day-by-day schedule editing; they reflect the current seeded prototype.

Data-integrity recommendations are: enforce date NOT NULL; use a date type; add non-negative checks for measurements and completion counts; validate realistic allowed ranges in server code; define constraints for required profile fields; store only password hashes; use a separate response DTO without the credential field; add an explicit migration history; and model plan/schedule identity with foreign keys rather than relying on table

suffix selection. Any schema change should be reviewed against existing data and deployed through a reversible migration process.

7 Module Design and Implementation Methodology

7.1 Module Inventory and Effort Estimate

Effort below is an illustrative student-work estimate in person-days for a clean, tested delivery. It is not a measurement of historical effort or a commitment. The baseline totals 36 person-days across overlapping calendar weeks; testing and integration are included as distinct tasks to reduce hidden effort.

Table 7: Module effort estimate and implementation status

ID	Module	Responsibility and process logic	Effort	Status
M1	Identity and session	Validate signup fields; persist profile; verify login; issue/validate token; clear session on logout. Replace plaintext storage before production.	4 days	Partial
M2	Profile and dashboard	Load profile and current metrics; derive visual progress against target values; render baseline and daily summary.	3 days	Implemented
M3	Plan catalogue	Map profile categories to cardio, strength, exercise, and diet seeds; serialize targets for the frontend.	5 days	Implemented, rule-based
M4	Daily logging	Accept water, steps, and completion values; validate; upsert by owner/date; return operation result.	4 days	Partial
M5	Date history/report	Fetch one date, combine stored values with targets, render progress. Add export only as a proposal.	3 days	In-app view
M6	API and persistence	Route requests to handlers; query PostgreSQL through the interface and GORM; migrate and seed tables.	5 days	Implemented
M7	UI and responsive workflow	Load templates, route navigation, build accessible forms, send JSON requests, handle empty/error states.	5 days	Implemented
M8	Verification and security	Add unit/integration tests, ownership checks, validation, response minimization, safe credential storage, and operational review.	5 days	Recommended
M9	Integration and documentation	Compose local services, configure deployment, verify end-to-end cases, maintain this synopsis and API/schema notes.	2 days	Partial
		Total estimated effort	**36 days**	

7.2 Process Logic by Module

7.2.1 M1: Identity and session

On signup, the browser checks that the two password entries match, calculates BMI from height and weight, and posts a profile object. The API checks whether the username exists and inserts a row. On login, the API loads the user, compares the submitted password, creates an HS256 JWT with an expiry, and returns the token. The browser stores token and username in localStorage, requests profile data, and clears those entries on logout. Proposed secure logic hashes passwords with a maintained adaptive algorithm, uses explicit request and response types, applies rate limits, verifies token algorithm and expiry, and obtains the requested owner from the validated token claim.

7.2.2 M2 and M3: Profile, dashboard, and plan catalogue

The browser loads the profile by username and renders identity and baseline values. It loads current activity and target information concurrently for the dashboard. Plan selection converts profile BMI and age into category identifiers, reads a map and the corresponding seeded schedule/diet catalogue, then serializes selected values as JSON. BMI is stored as an integer in the current user table even though the browser calculates a decimal and rounds it before submission. Category thresholds and units should be reviewed and represented with explicit domain types before use as health guidance.

7.2.3 M4 and M5: Activity logging and history

The statistics screen loads current daily values and plan targets; it renders numerical fields for water, steps, and each supported exercise. Submitted values become a JSON object with names matching the workout model. The API writes or updates a row for username and the package-level current date. The reports view accepts a date, loads the record and corresponding targets, and displays progress bars. Current reports are interactive single-date screens. CSV/PDF export, weekly trends, and comparative analytics are proposals only.

7.2.4 M6–M9: API, persistence, verification, and delivery

The API registers route handlers and wraps selected routes in JWT middleware and a CORS handler. The database implementation connects using environment variables, runs AutoMigrate, seeds lookup and plan rows, and implements reads and writes via GORM. The frontend template loader replaces view content, attaches event handlers at the application root, and manages request errors in the UI. A complete implementation methodology should use iterative requirements, schema and API design, small vertical slices, automated tests, security review, deployment verification, and user acceptance checks. The repository contains no test files at the time of inspection; test coverage is a required next phase, not a completed result.

7.3 Data Structures

Table 8: Principal data structures by layer

Layer	Structure	Use
Browser	DOM templates and plain JavaScript objects	View state, profile cache, plan cache, JSON payloads, and dynamic form data.
Browser	localStorage string keys	Current token and username. Recommended replacement: a server-managed secure, HttpOnly session where architecture permits.
API	Go structs	User, Workoutplan, Weekday, DietPlan, and response structures; JSON tags map API fields.
API	maps and slices	JSON target maps, schedule arrays, exercise lists, and dynamically assembled daily workout summaries.
API	JWT claims	Username and standard expiry claim signed using HS256.
Persistence	PostgreSQL relations	User rows, lookup tables, seeded plan catalogues, and one user/date activity record.

7.4 Reports Likely to Be Generated

- **Implemented:** the dashboard's current-day hydration, steps, and workout progress summary.
- **Implemented:** a selected-date in-app activity summary showing water, steps, target completion, and exercise counts.
- **Implemented:** profile baseline display and plan screens for cardio, strength, and meal suggestions.
- **Proposed:** weekly/monthly trend summaries, CSV export, print-ready PDF, and goal-completion summaries. These are not generated by current source.

8 Project Planning and Scheduling

8.1 Planning Assumptions

The schedule is relative, not calendar-dated, because no academic deadline was supplied. It assumes one student, access to a laptop and internet, and a project span of eight weeks. The workday estimates in Table 7 inform the plan but do not claim measured productivity. Risk review and security work are included explicitly.

8.2 Gantt Schedule

The figure shows a proposed week-based plan. A filled segment marks active work. Work packages overlap only where requirements and interface contracts permit; final integration and review depend on the earlier implementation tasks.

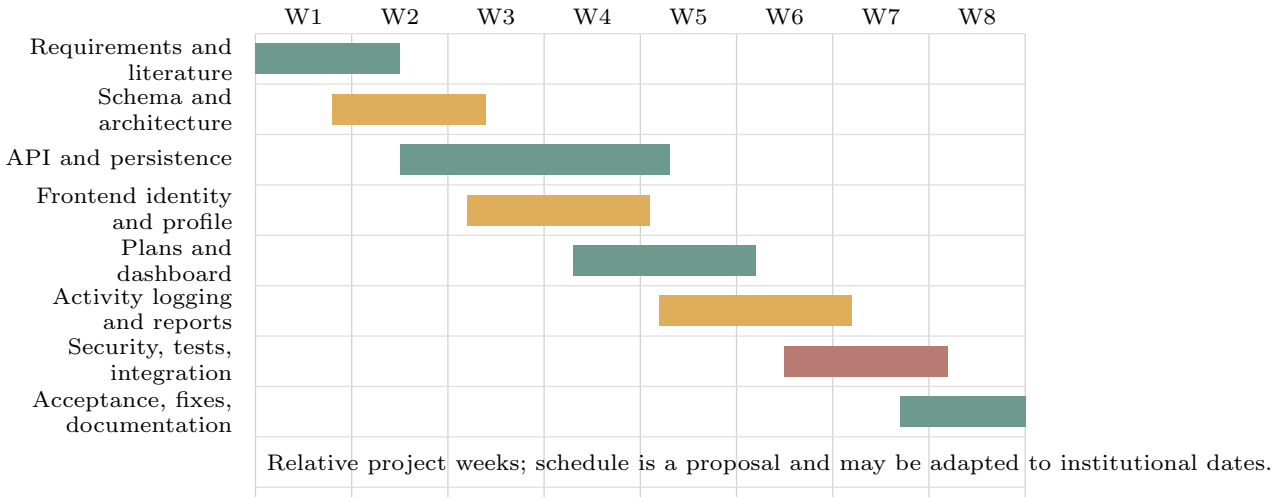


Figure 8: Proposed eight-week Gantt schedule.

8.3 PERT Network

The network expresses precedence, not a claim of statistically estimated duration. Durations use working days from the illustrative effort estimate. Activities A–H represent: requirements (3), schema/architecture (5), API/data (9), frontend (8), plan and report integration (4), test/security (5), acceptance fixes (2), and final documentation (2). The longest displayed path is A–B–C–D–E–F–G–H, which totals 38 working days under the simplifying assumption that these activities are fully sequential. Some frontend and backend work can proceed concurrently after interface agreements, so elapsed calendar time is shorter in the Gantt plan.

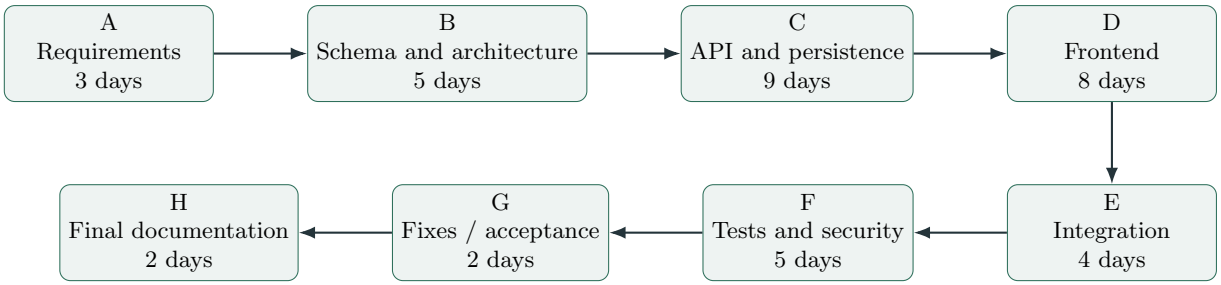


Figure 9: PERT precedence network for the proposed baseline schedule.

Table 9: Project risks and practical responses

Risk	Effect	Mitigation
Plaintext credentials and overexposed profile response	Account compromise and unnecessary disclosure.	Hash passwords, use explicit response DTOs, remove credential data from logs and responses, and test regressions.
Missing owner-to-token binding	Cross-account access to profile or activity.	Use the authenticated subject for every data operation; reject mismatched identifiers; add authorization tests.
Stale date/day globals	Data may be recorded or selected under an incorrect day after process uptime crosses midnight.	Compute date at request time in an explicit timezone and pass it through business logic.
Weak server-side validation	Invalid profile and activity values may enter storage.	Validate JSON, allowed fields, dates, positive ranges, and units server-side; add database constraints.
Unversioned automatic migrations	Schema changes may be hard to review or safely roll back.	Introduce versioned migrations, backups, staging verification, and a documented rollback procedure.
No automated tests found	Regressions and security failures may go undetected.	Add unit, database integration, API authorization, and browser workflow tests.

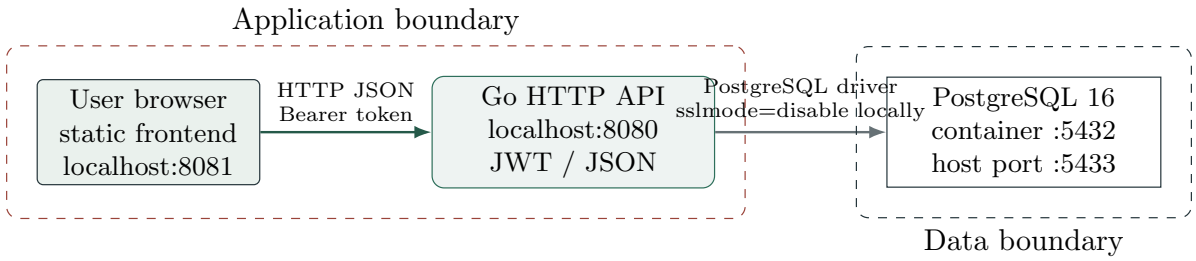


Figure 10: Current local network architecture and trust boundaries.

8.4 Risks and Mitigations

9 Network Architecture and Security

9.1 Overall Network Architecture

For local development, browser-to-API traffic uses HTTP and the API-to-database DSN uses disabled SSL. The compose file publishes PostgreSQL on host port 5433 and maps it to container port 5432. CORS allows localhost frontend origins by default; CORS is a browser policy and is not authentication. A deployment design should serve the frontend and API through TLS, restrict database exposure to the application network, rotate secrets, apply least privilege, and separate development and production configuration. The actual production network architecture is not established by the repositories and is therefore a proposal, not a deployed fact.

9.2 Security Mechanisms: Current State and Required Work

Table 10: Security controls, gaps, and recommendations

Area	Current source evidence	Assessment and recommended action
Authentication	Login creates an HS256 token with an expiry; most data routes are wrapped by bearer-token middleware.	Useful prototype control. Hash passwords using a maintained adaptive scheme; add rate limiting, consistent token validation, and test expiry and invalid-token behavior.
Authorization	Middleware checks token validity and stores a username claim in request context. Handlers generally take the requested username from the URL.	Subject-to-resource ownership is not consistently enforced. Derive owner from verified claims and reject cross-account access for every profile and log operation.
Credential confidentiality	Password is stored as a string and compared directly. Profile response serializes the full user model.	Critical gap. Migrate to password hashes and return a credential-free response type. Treat existing data as compromised if it ever contained real passwords.
Browser session	Token and username are stored in localStorage.	A successful script injection can expose stored tokens. Prefer a secure, HttpOnly, SameSite cookie session or otherwise apply a considered token-storage design and strong content security controls.
Transport	Local examples use HTTP; database DSN sets sslmode=disable.	Acceptable only for isolated local development. Use TLS for client/API and database links where the deployment threat model requires it.
Secrets	JWT secret is required from environment; database defaults include development credentials.	Do not use defaults in deployment. Store secrets in a managed secret facility, rotate them, and never log them.
CORS and HTTP	CORS origins are configurable; credentials are disabled. Generic handlers do not consistently declare allowed methods.	Keep the allowlist narrow, define explicit methods and content types, and do not treat CORS as an access-control mechanism.
Validation and privacy	Browser controls constrain some inputs; server handlers decode model structs. Request logging prints request/token-related details in parts of the code.	Validate strictly on the server, minimize responses, remove secrets/tokens from logs, define retention/deletion policies, and test invalid payloads.
Database integrity	Primary keys, unique indexes, and several foreign keys are declared; workout owner FK cascades.	Add date NOT NULL and range checks; inspect actual constraints; adopt reviewed migrations and backups.

9.3 Security Requirements by Level

At the application level, every route must authenticate where required, authorize the requested user from token claims, reject unknown or unexpected fields, validate numeric ranges and date formats, and return minimal JSON. At the database level, credentials must be least-privileged, production connections encrypted where applicable, backups protected, constraints enforced, and user data deleted according to a documented retention policy. At the transport level, use TLS and secure browser session settings. At the operational level, rotate JWT and database secrets, monitor failed logins and errors, patch dependencies, maintain backups, and test restoration. These are requirements for the next secure release; they are not claimed as all present controls.

10 Future Scope, Conclusion, and Bibliography

10.1 Future Scope and Enhancements

The following enhancements are realistic but outside the present implementation:

1. Complete the security baseline: password hashing and migration, token subject authorization, response DTOs, TLS deployment, secret management, rate limits, and threat-focused automated tests.
2. Replace category-specific repeated plan tables with a normalized catalogue of plans, exercises, weekday assignments, targets, and diet entries, with explicit foreign keys and versioned migrations.
3. Improve data quality with decimal BMI values, explicit units, server-side validation, a documented timezone policy, and user-editable profile details.
4. Provide trend charts, weekly/monthly summaries, and exportable reports after defining what data users may export and how it is protected.
5. Add optional wearable integration only after defining consent, data provenance, synchronization, deduplication, and deletion behavior.
6. Add a coaching service only after privacy review and clear clinical boundaries. Until then, the chat remains a UI preview.
7. Add accessibility review, browser workflow automation, API contract tests, structured logging without secrets, monitoring, backups, and a deployment runbook.

10.2 Conclusion

Formwell is a full-stack fitness tracking demonstrator with a browser interface, Go API, JWT bearer authentication, and PostgreSQL persistence. Its strongest project value is the integrated profile-to-plan-to-activity workflow and the opportunity to demonstrate relational modeling, client-server communication, and application modularity. The current implementation supports meaningful prototype functionality, but several security and integrity gaps are material. The report records those gaps instead of presenting recommended controls as finished features. With secure credential handling, strict ownership checks, schema validation, automated tests, and reviewed migrations, the system can provide a sounder basis for an academic implementation and future expansion.

References

[1] World Health Organization. *WHO Guidelines on Physical Activity and Sedentary Behaviour*. Geneva: World Health Organization; 2020. ISBN 978-92-4-001512-8. Available at: `https://www.who.int/publications/i/item/9789240015128`.

[2] Jones, M., Bradley, J., and Sakimura, N. *JSON Web Token (JWT)*. RFC 7519. Internet Engineering Task Force; May 2015. `https://www.rfc-editor.org/rfc/rfc7519`.

[3] GORM Authors. *Migration*. GORM documentation. `https://gorm.io/docs/migration.html`.

[4] PostgreSQL Global Development Group. *Constraints*. PostgreSQL 16 Documentation, Section 5.4. `https://www.postgresql.org/docs/16/ddl-constraints.html`.

[5] OWASP Cheat Sheet Series. *Password Storage Cheat Sheet*. `https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html`.

[6] Formwell frontend source repository. *README and application pages*. Local project source: `frontend-fitness/README.md`, `app.js`, and `pages/`.

[7] Fitness Tracker API source repository. *README, Go handlers, models, and GORM database implementation*. Local project source: `backend-fitness/README.md`, `main.go`, `handler/handler.go`, `models/models.go`, and `db/`.